DETAILS

NAME: LINGAM.KAVYA SRI

SECTION: CSE 1

ROLL NO: 26000035

PYTHON ACTIVITY 1

CASE STUDIES 21-30

21.Traffic Violation Penalty Calculator

22.Laboratory Chemical Inventory Validation

23.Student Timetable Conflict Detection

24.Courier Charge Optimization

25.Progressive Water Tariff Analysis

26.Nested Loop Examination Seating Generator

27.Advanced Numeric Pattern Generator

28.Repeated Transaction Detection

29.Password Policy Validation Engine

30.Student Email and Identifier Validation

CASE STUDY-21: Traffic Violation Penalty Calculator

1.Problem statement

To develop a Python program to calculate traffic violation fines based on vehicle category, offence type, and repeat offences.

2. Objectives

- To identify the type of traffic violation.
- To calculate the applicable fine.
- To check whether the offence is repeated.
- To display the final penalty.

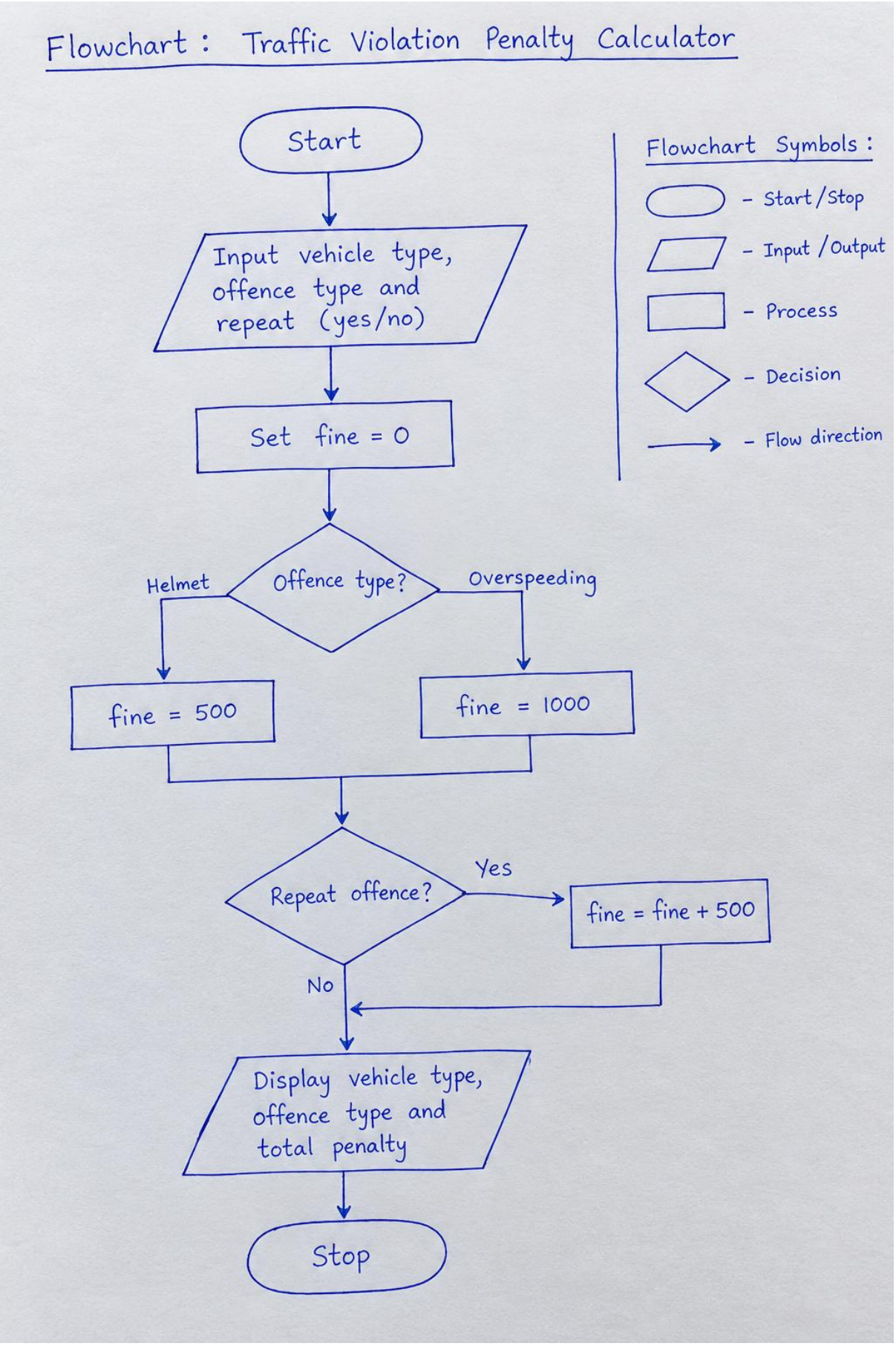
3. Assumptions

- Two-wheeler and car are the vehicle categories.
- Helmet violation fine is ₹500.
- Overspeeding fine is ₹1,000.
- Repeat offences have an additional fine of ₹500.
- These are hypothetical values for this assignment.

4. Algorithm

1. Start.
2. Read vehicle category and offence type.
3. Check the offence and calculate the fine.
4. Check whether it is a repeat offence.
5. Add the extra fine if repeated.
6. Display the total penalty.
7. Stop

5.Flow chart


Flowchart : Traffic Violation Penalty Calculator
Start
Input vehicle type, offence type and repeat (yes/no)
Set fine = 0
Offence type?
Helmet
Overspeeding
fine = 500
fine = 1000
Repeat offence?
Yes
fine = fine + 500
No
Display vehicle type, offence type and total penalty
Stop
Flowchart Symbols :
- Start/Stop
- Input /Output
- Process
- Decision
- Flow direction

6. Pseudocode

START

INPUT vehicle, offence, repeat

IF offence is helmet violation

  fine = 500

ELSE IF offence is overspeeding

  fine = 1000

ELSE

  fine = 0

IF repeat is yes

  fine = fine + 500

PRINT fine

STOP

7.Python Program

```
vehicle = input("Enter vehicle type: ")
offence = input("Enter offence (helmet/over speeding): ")
repeat = input("Is it a repeat offence? (yes/no): ")

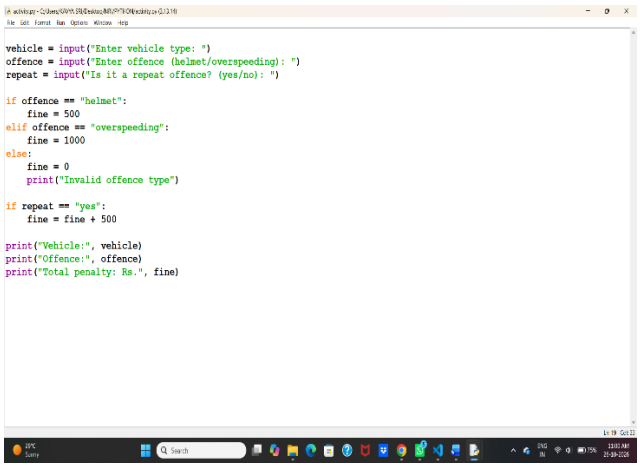
if offence == "helmet":
  fine = 500
elif offence == "over speeding":
  fine = 1000
else:
  fine = 0
  print("Invalid offence type")

if repeat == "yes":
  fine = fine + 500

print("Vehicle:", vehicle)
print("Offence:", offence)
print("Total penalty: Rs.", fine)
```

8.Test cases

Offence	Repeat?	Expected fine
helmet	no	₹500
overspeeding	no	₹1,000
helmet	yes	₹1,000

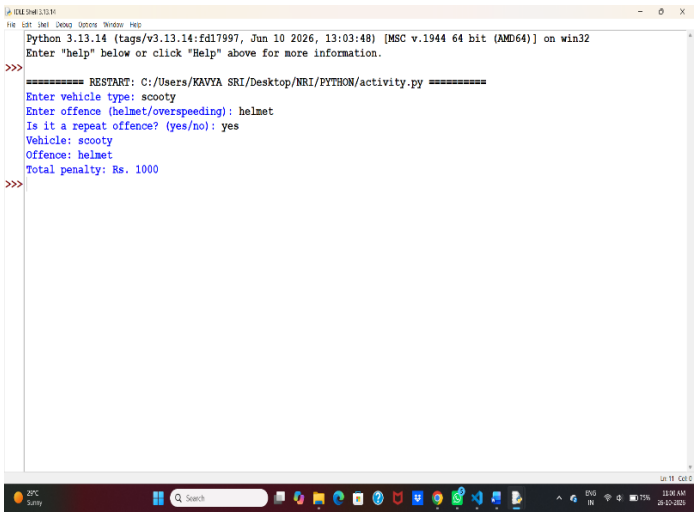


```
vehicle = input("Enter vehicle type: ")
offence = input("Enter offence (helmet/overspeeding): ")
repeat = input("Is it a repeat offence? (yes/no): ")

if offence == "helmet":
    fine = 500
elif offence == "overspeeding":
    fine = 1000
else:
    fine = 0
    print("Invalid offence type")

if repeat == "yes":
    fine = fine + 500

print("Vehicle:", vehicle)
print("Offence:", offence)
print("Total penalty: Rs.", fine)
```



```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/KAVYA SRI/Desktop/NRI/PYTHON/activity.py ==========
Enter vehicle type: scooty
Enter offence (helmet/overspeeding): helmet
Is it a repeat offence? (yes/no): yes
Vehicle: scooty
Offence: helmet
Total penalty: Rs. 1000
>>>
```

9. Sample Output

Enter vehicle type: car

Enter offence (helmet/overspeeding): overspeeding

Is it a repeat offence? (yes/no): no

Vehicle: car

Offence: overspeeding

Total penalty: Rs. 1000

10. Conclusion

This program calculates traffic violation penalties using conditional statements and displays the total fine based on the offence and repeat-offence status.

CASE STUDY 22: Laboratory Chemical Inventory Validation

1. Problem Statement

To develop a Python program to check laboratory chemical inventory records using storage categories, maximum quantity limits, and compatibility rules, and identify invalid records.

2. Objectives

- To store chemical names and quantities.
- To check maximum quantity limits.
- To validate storage categories.
- To display valid and invalid inventory records.

3. Assumptions

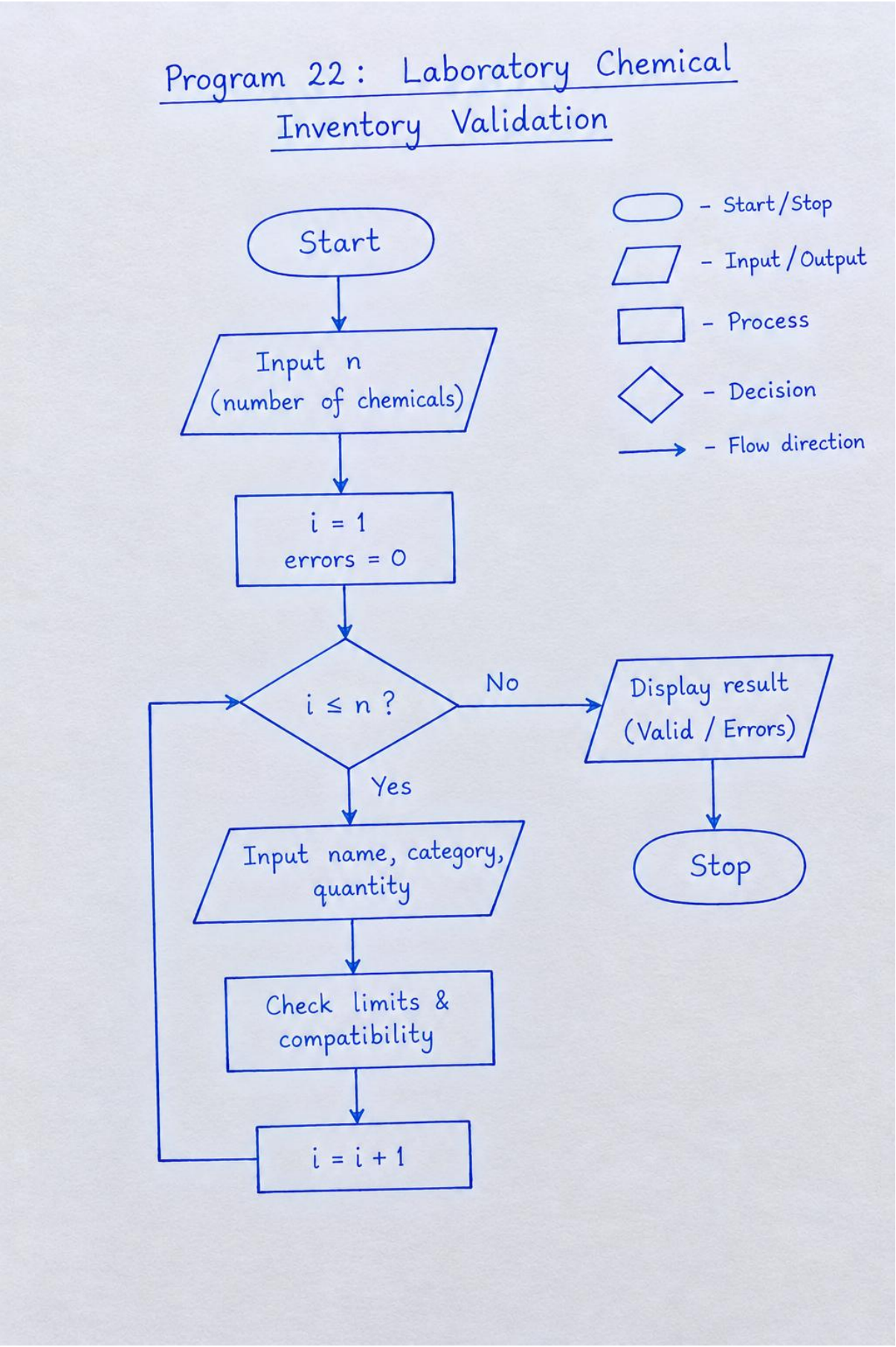
For this example, the following are hypothetical assignment rules:

- Maximum permitted quantity is 100 units per chemical.
- Each chemical must have a valid storage category.
- Chemicals stored in incompatible categories are flagged for review.
- The example uses fictional chemical labels and simplified rules, not real laboratory safety guidance.

4. Algorithm

1. Start.
2. Input the number of chemical records.
3. Enter each chemical's name, category, and quantity.
4. Check whether the quantity exceeds the allowed limit.
5. Check whether any two categories are incompatible under the given sample rules.
6. Display any errors found.
7. If no errors are found, display “Inventory is valid.”
8. Stop.

5.flow chart


Program 22 : Laboratory Chemical Inventory Validation
Start
Input n (number of chemicals)
i = 1 errors = 0
i ≤ n ?
No
Yes
Display result (Valid / Errors)
Stop
Input name, category, quantity
Check limits & compatibility
i = i + 1
- Start / Stop
- Input / Output
- Process
- Decision
- Flow direction

6. Pseudocode

```
START
INPUT number of chemicals
SET errors = 0

FOR each chemical
  INPUT name, category, quantity
  CHECK quantity limit
  CHECK compatibility rules
  IF any rule is violated
    DISPLAY error
    SET errors = errors + 1
  END IF
END FOR

IF errors = 0
  DISPLAY "Inventory is valid"
END IF

STOP
```

7. Python program

```
n = int(input("Enter number of chemicals: "))
errors = 0

for i in range(n):
    name = input("Enter chemical name: ")
    category = input("Enter category: ")
    qty = int(input("Enter quantity: "))

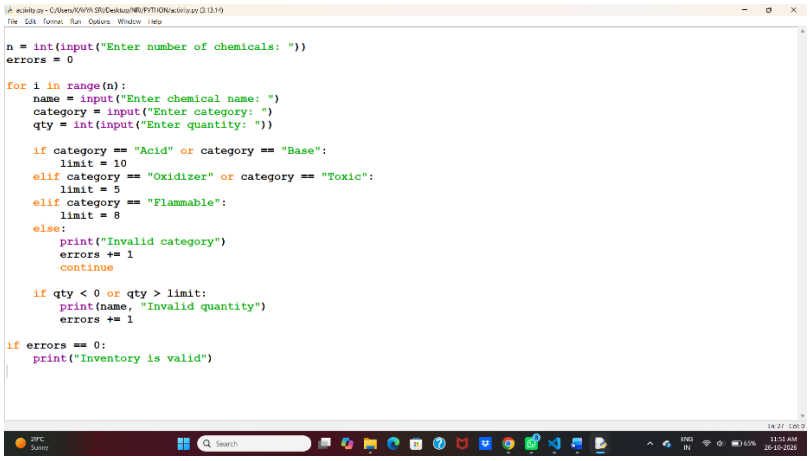
    if category == "Acid" or category == "Base":
        limit = 10
    elif category == "Oxidizer" or category == "Toxic":
        limit = 5
    elif category == "Flammable":
        limit = 8
    else:
        print("Invalid category")
        errors += 1
        continue

    if qty < 0 or qty > limit:
        print(name, "Invalid quantity")
        errors += 1

if errors == 0:
    print("Inventory is valid")
```

8. Test Cases

Test case	Input	Expected result
1	Sample category Toxic, quantity 3	Inventory is valid
2	Sample category Acid, quantity 15	Quantity exceeds limit
3	Invalid category Water, quantity 2	Invalid category
4	Sample category Base, quantity -2	Invalid quantity



```
n = int(input("Enter number of chemicals: "))
errors = 0

for i in range(n):
    name = input("Enter chemical name: ")
    category = input("Enter category: ")
    qty = int(input("Enter quantity: "))

    if category == "Acid" or category == "Base":
        limit = 10
    elif category == "Oxidizer" or category == "Toxic":
        limit = 5
    elif category == "Flammable":
        limit = 8
    else:
        print("Invalid category")
        errors += 1
        continue

    if qty < 0 or qty > limit:
        print(name, "Invalid quantity")
        errors += 1

if errors == 0:
    print("Inventory is valid")
```



```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/KAVYA SRI/Desktop/NRI/PYTHON/activity.py ==========
Enter number of chemicals: 1
Enter chemical name: h2o2
Enter category: acid
Enter quantity: 1
Invalid category
>>>
```

9. **Sample Output**

   Enter number of chemicals: 1
   Enter chemical name: SampleA
   Enter category: Toxic
   Enter quantity: 3
   Inventory is valid

10. Conclusion

This program uses Python lists, tuples, loops, conditional statements, and a dictionary to validate hypothetical chemical inventory records and report errors.

CASE STUDY 23: Student Timetable Conflict Detection

1. Problem Statement

To develop a Python program to detect timetable clashes by comparing course schedules, time periods, and room availability.

2. Objectives

- To store course schedules using tuples.
- To identify overlapping time periods.
- To check whether rooms are available.
- To display timetable conflicts.

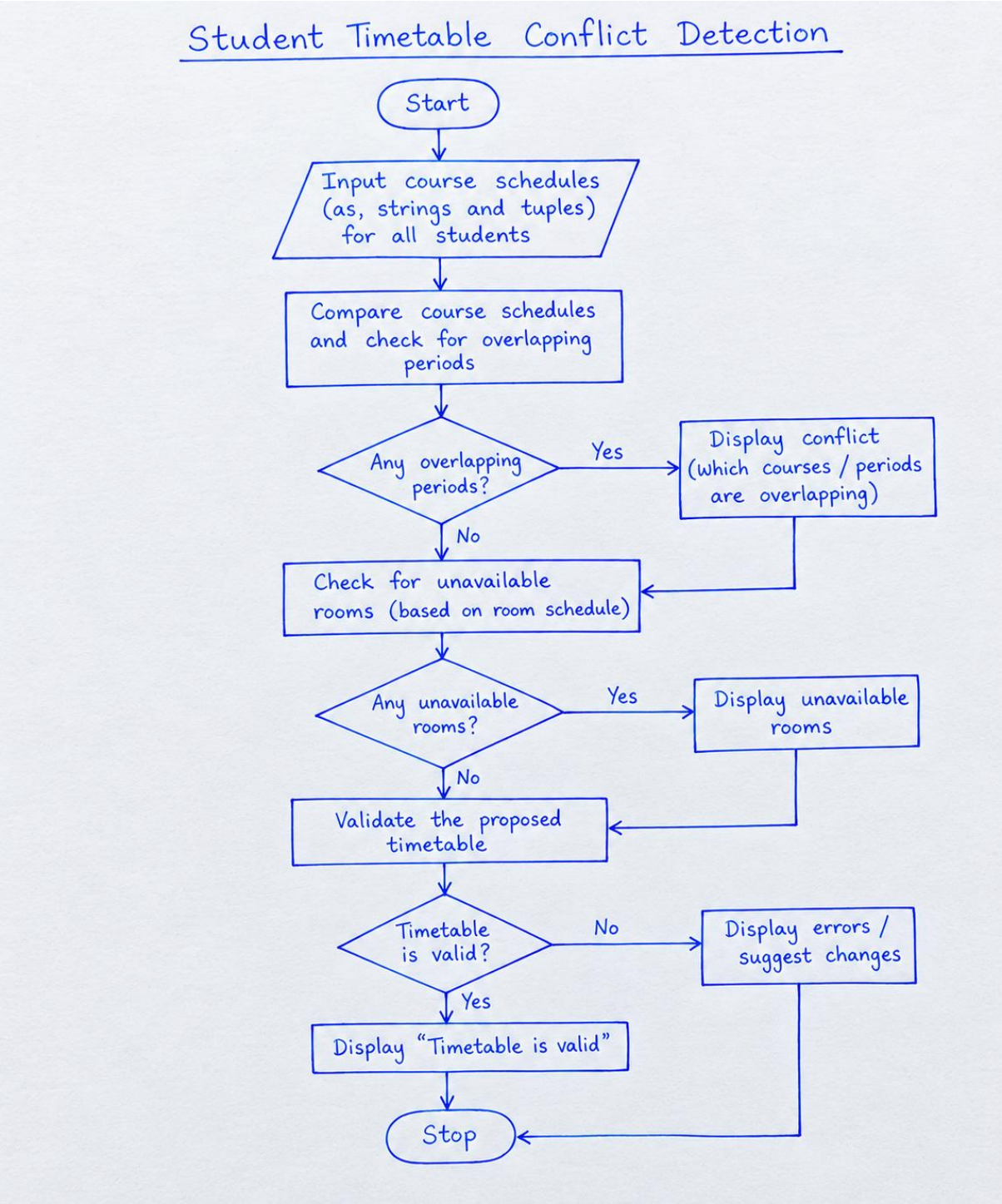
3. Assumptions

- Each class has a course name, day, start time, end time, and room.
- Classes scheduled on the same day with overlapping times are considered a conflict.
- Two classes cannot use the same room at overlapping times.

4. Algorithm

1. Start.
2. Store course schedules in a list of tuples.
3. Compare each pair of schedules.
4. Check whether the day and time periods overlap.
5. Check whether the same room is assigned.
6. Display any conflicts found.
7. Stop.

5.Flow chart


Student Timetable Conflict Detection
Start
Input course schedules (as, strings and tuples) for all students
Compare course schedules and check for overlapping periods
Any overlapping periods?
Yes
Display conflict (which courses / periods are overlapping)
No
Check for unavailable rooms (based on room schedule)
Any unavailable rooms?
Yes
Display unavailable rooms
No
Validate the proposed timetable
Timetable is valid?
No
Display errors / suggest changes
Yes
Display "Timetable is valid"
Stop

6. Pseudocode

START

Store course schedules

FOR each pair of classes

IF same day AND times overlap

IF same room

DISPLAY room conflict

ELSE

DISPLAY timetable conflict

STOP

7.Python Program

```
classes = [
  ("Python", "Monday", 9, 11, "R1"),
  ("Maths", "Monday", 10, 12, "R1"),
  ("English", "Tuesday", 9, 10, "R2")
]

conflict = False

for i in range(len(classes)):
  for j in range(i + 1, len(classes)):
    c1 = classes[i]
    c2 = classes[j]

    if c1[1] == c2[1] and c1[2] < c2[3] and c2[2] < c1[3]:
      if c1[4] == c2[4]:
```

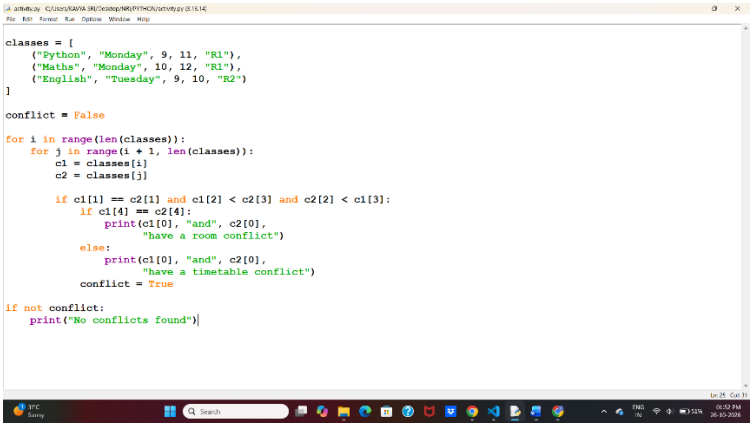
```
            print(c1[0], "and", c2[0],
                  "have a room conflict")
        else:
            print(c1[0], "and", c2[0],
                  "have a timetable conflict")
        conflict = True

if not conflict:
    print("No conflicts found")
```

8. Test Cases and Output

8.Test cases

Course 1	Course 2	Result
Python, Monday 9–11	Maths, Monday 10–12	Room conflict
Python, Monday 9–11	English, Tuesday 9–10	No conflict



```
classes = [
    ("Python", "Monday", 9, 11, "R1"),
    ("Maths", "Monday", 10, 12, "R1"),
    ("English", "Tuesday", 9, 10, "R2")
]

conflict = False

for i in range(len(classes)):
    for j in range(i + 1, len(classes)):
        c1 = classes[i]
        c2 = classes[j]

        if c1[1] == c2[1] and c1[2] < c2[3] and c2[2] < c1[3]:
            if c1[4] == c2[4]:
                print(c1[0], "and", c2[0],
                      "have a room conflict")
            else:
                print(c1[0], "and", c2[0],
                      "have a timetable conflict")
            conflict = True

if not conflict:
    print("No conflicts found")
```



9.sampleoutput:

Python and Maths
have a room conflict

10. Conclusion:

This program uses lists, tuples, nested loops, and conditional statements to detect overlapping class schedules and room conflicts.

CASE STUDY 24: Courier Charge Optimization

1. Problem Statement

To develop a Python program to calculate courier charges based on parcel weight, distance, delivery type, and restrictions, and select the cheapest eligible service.

2. Objectives

- To calculate courier charges.
- To check parcel weight and delivery type.
- To compare different courier services.
- To select the cheapest eligible service.

3. Assumptions

- Standard delivery costs ₹50 plus ₹10 per kg.
- Express delivery costs ₹100 plus ₹15 per kg.
- Maximum parcel weight is 20 kg.
- These are hypothetical charges for this assignment.

4. Algorithm

1. Start.
2. Input parcel weight and distance.
3. Check whether the weight is valid.
4. Calculate standard and express charges.
5. Compare both charges.
6. Display the cheapest service.
7. Stop.

5. Pseudocode

```
START

INPUT weight, distance


IF weight <= 0 OR weight > 20

   PRINT "Invalid weight"
```

ELSE

standard = 50 + weight * 10

express = 100 + weight * 15

IF standard < express

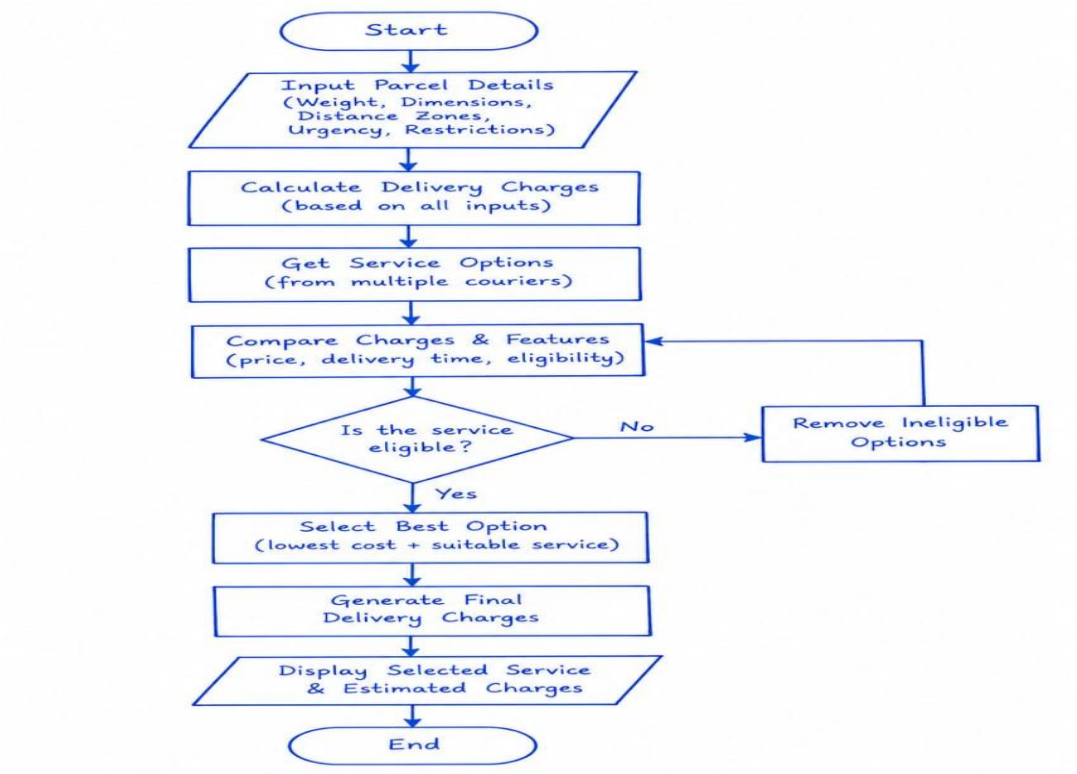
PRINT standard service

ELSE

PRINT express service

STOP

6. Flow chart



7. Python Program

```
weight = float(input("Enter parcel weight: "))

distance = float(input("Enter distance: "))


if weight <= 0 or weight > 20 or distance <= 0:
```

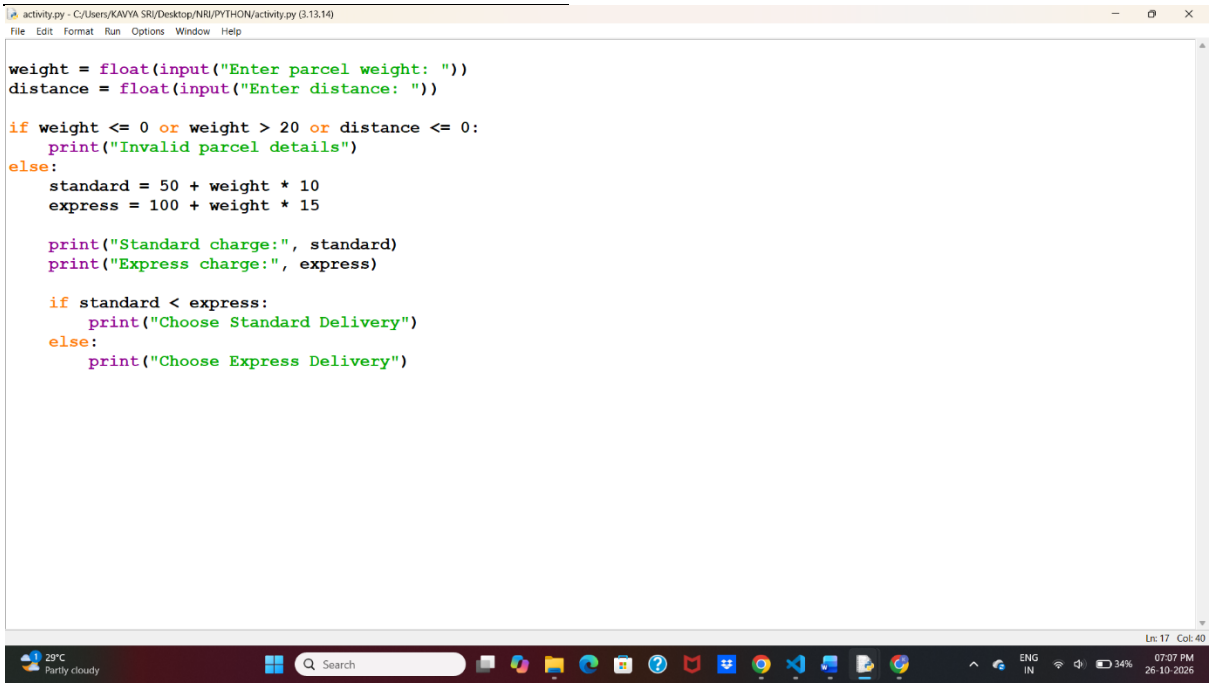
```
    print("Invalid parcel details")
else:
    standard = 50 + weight * 10
    express = 100 + weight * 15

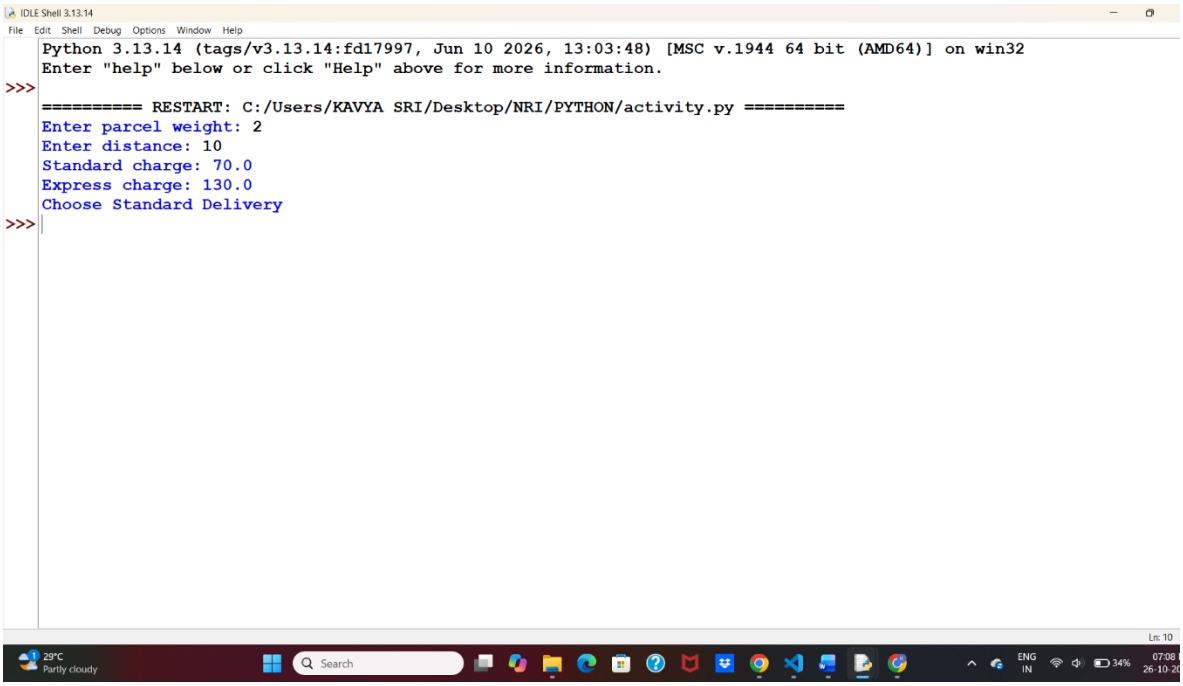
    print("Standard charge:", standard)
    print("Express charge:", express)

    if standard < express:
        print("Choose Standard Delivery")
    else:
        print("Choose Express Delivery")
```

8.Test Cases

Weight	Distance	Expected result
2 kg	10 km	Standard: ₹70
5 kg	20 km	Standard: ₹100
25 kg	10 km	Invalid parcel details





9.Sample output

Enter parcel weight: 2

Enter distance: 10

Standard charge: 70.0

Express charge: 130.0

Choose Standard Delivery

10. Conclusion

This program uses Python conditional statements and arithmetic operators to calculate courier charges and select the cheaper delivery service.

CASE STUDY 25: Progressive Water Tariff Analysis

1. Problem Statement

To develop a Python program to calculate water bills based on the quantity of water consumed using progressive tariff rates.

2. Objectives

- To calculate water consumption charges.
- To apply different rates for different consumption levels.
- To display the total water bill.

3. Assumptions

- First 100 units: ₹2 per unit.
- Next 100 units: ₹3 per unit.
- Above 200 units: ₹5 per unit.

Note: These are hypothetical rates for the assignment.

4. Algorithm

1. Start.
2. Input water consumption units.
3. If units are within 100, calculate at ₹2 per unit.
4. If units are between 101 and 200, calculate the first 100 units at ₹2 and remaining units at ₹3.
5. If units exceed 200, calculate the first 100 at ₹2, next 100 at ₹3, and remaining units at ₹5.
6. Display the total bill.
7. Stop.

5. Pseudocode

```
START

INPUT units


IF units < 0

   PRINT "Invalid units"
```

ELSE IF units <= 100

bill = units * 2

ELSE IF units <= 200

bill = 200 + (units - 100) * 3

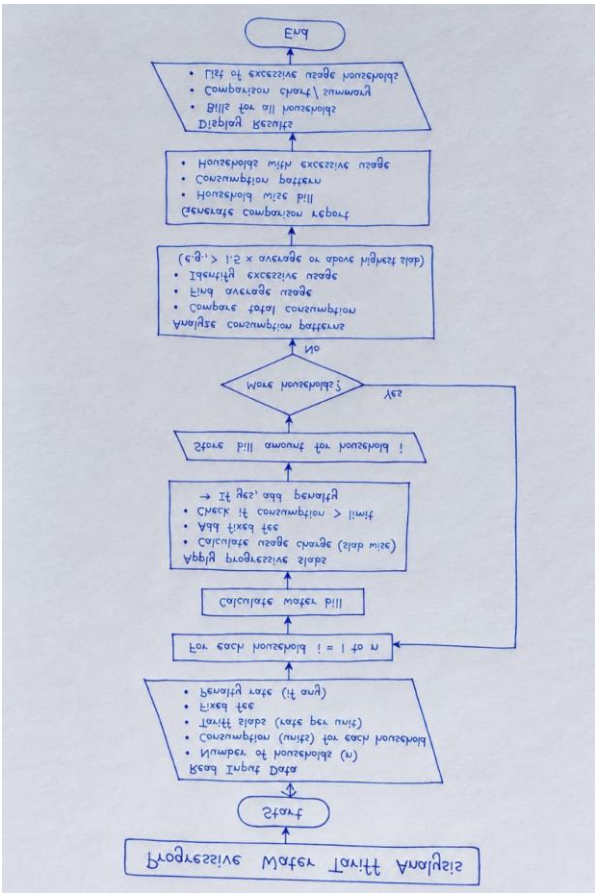
ELSE

bill = 500 + (units - 200) * 5

PRINT bill

STOP

6.flow chart



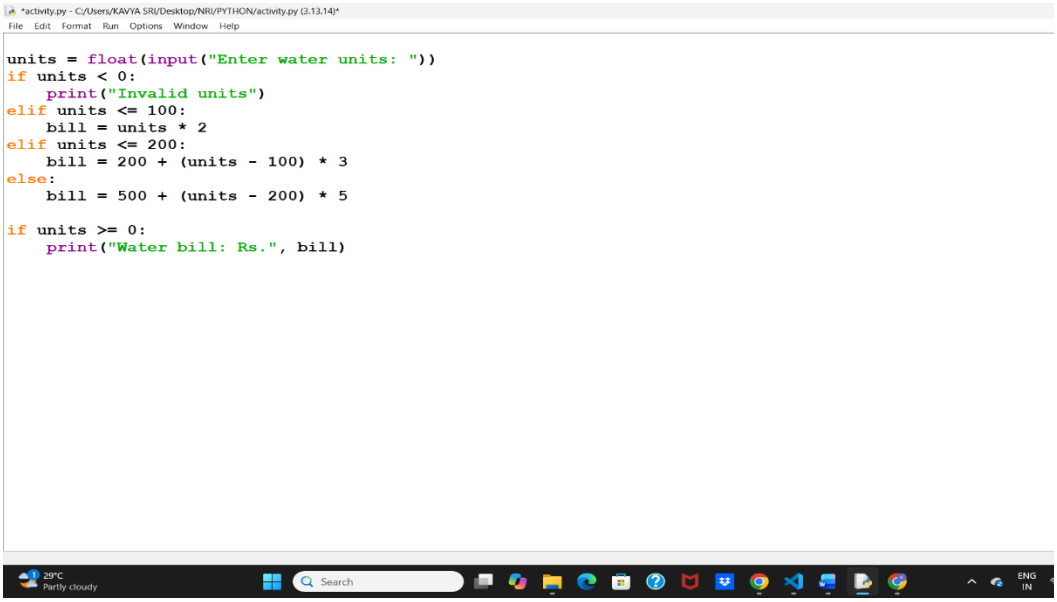
7.Python Program

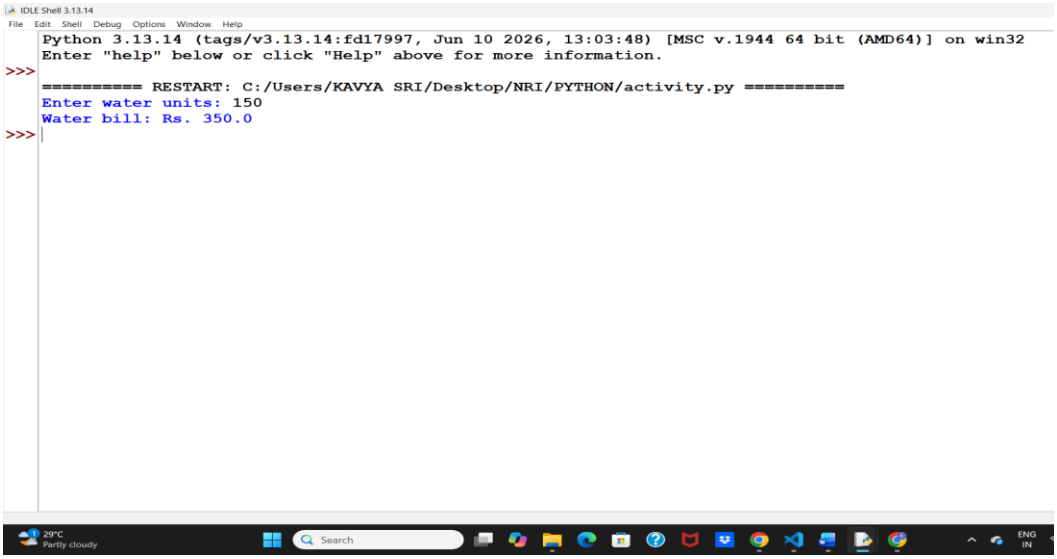
units = float(input("Enter water units: "))

```
if units < 0:
    print("Invalid units")
elif units <= 100:
    bill = units * 2
elif units <= 200:
    bill = 200 + (units - 100) * 3
else:
    bill = 500 + (units - 200) * 5
if units >= 0:
    print("Water bill: Rs.", bill)
```

8. Test Cases

Water units	**Bill**
50	₹100
150	₹350
250	₹750





9.Sample output

Enter water units: 150

Water bill: Rs. 350.0

10. Conclusion

This program calculates water bills using progressive tariff rates and Python conditional statements.

CASE STUDY 26: Nested Loop Examination Seating Generator

1. Problem Statement

To develop a Python program to generate an examination seating arrangement using rows, columns, and nested loops.

2. Objectives

- To create a seating arrangement.
- To display row and column numbers.
- To reserve selected seats.
- To use nested loops for seat generation.

3. Assumptions

- The classroom has 3 rows and 4 columns.
- Seat R2C2 is reserved.
- All other seats are available.

4. Algorithm

1. Start.
2. Set the number of rows and columns.
3. Use nested loops to visit each seat.
4. Check whether the seat is reserved.
5. Display the seat as reserved or available.
6. Stop.

5. Pseudocode

```
START

SET rows = 3, columns = 4

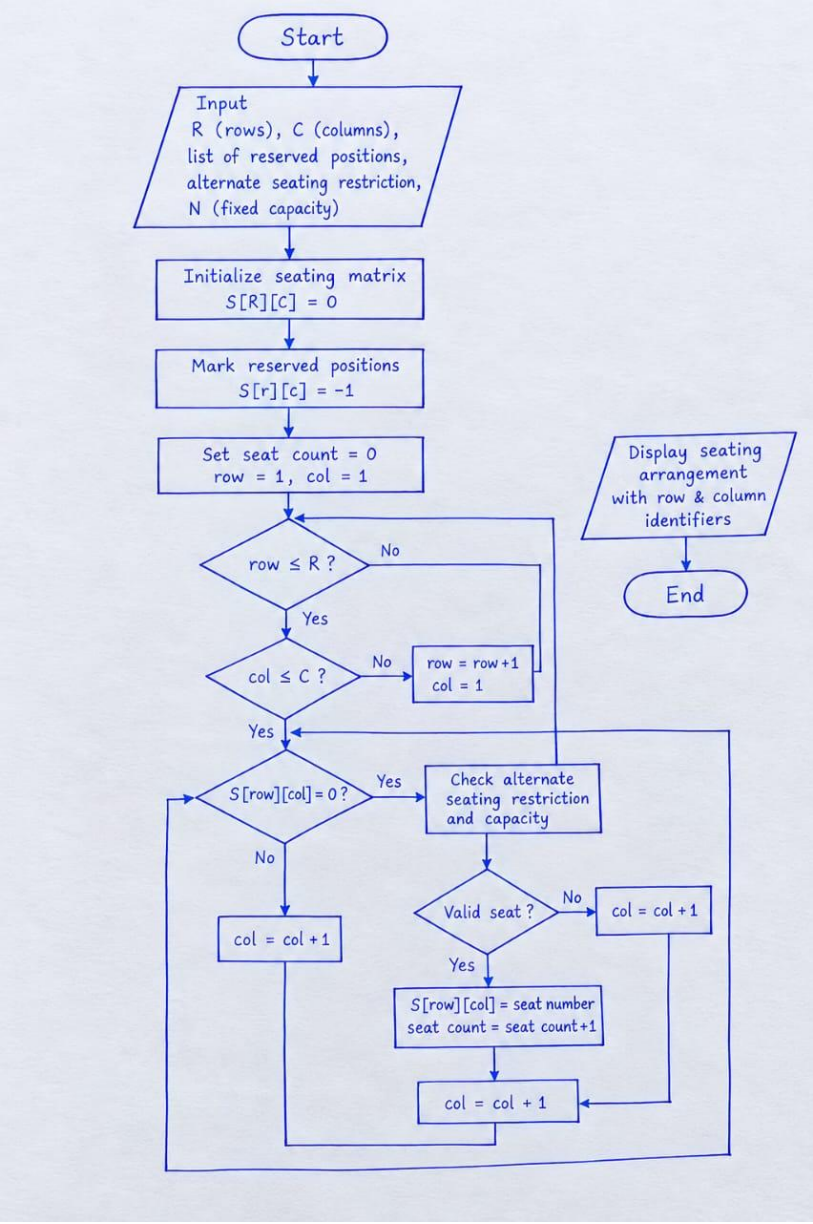

FOR each row
   FOR each column
      IF seat is reserved
         PRINT "Reserved"
```

ELSE

PRINT "Available"

STOP

6.flow chart



7. Python Program

rows = 3

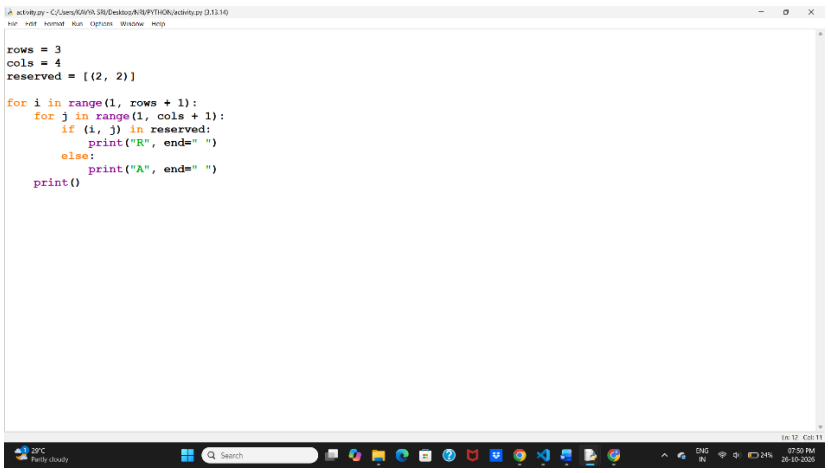
cols = 4

reserved = [(2, 2)]

```
for i in range(1, rows + 1):
    for j in range(1, cols + 1):
        if (i, j) in reserved:
            print("R", end=" ")
        else:
            print("A", end=" ")
    print()
```

8.Test Cases

Test Case	Input	Expected Output
1	Rows = 3, Columns = 4	3 rows and 4 columns display avuthayi
2	Seat R2C2	Reserved (R)
3	Other seats	Available (A)





9. Sample Output

A A A A

A R A A

A A A A

A = Available seat, R = Reserved seat.

10. Conclusion

This program uses nested loops, tuples, and conditional statements to generate an examination seating arrangement and identify reserved seats.

CASE STUDY 27: Advanced Numeric Pattern Generator

1. Problem Statement

To develop a Python program that generates numeric patterns using loops based on the number of rows entered by the user.

2. Objectives

- To generate numeric patterns.
- To use loops and nested loops.
- To improve logical thinking.

3. Assumptions

- The user enters the number of rows.
- The number of rows must be greater than zero.
- The program prints a simple number pattern.

4. Algorithm

1. Start.
2. Input the number of rows.
3. Check whether the number is greater than zero.
4. Use nested loops to print the number pattern.
5. Display the pattern.
6. Stop.

5. Pseudocode

```
START

INPUT rows


IF rows <= 0

  PRINT "Invalid input"

ELSE

  FOR i = 1 TO rows

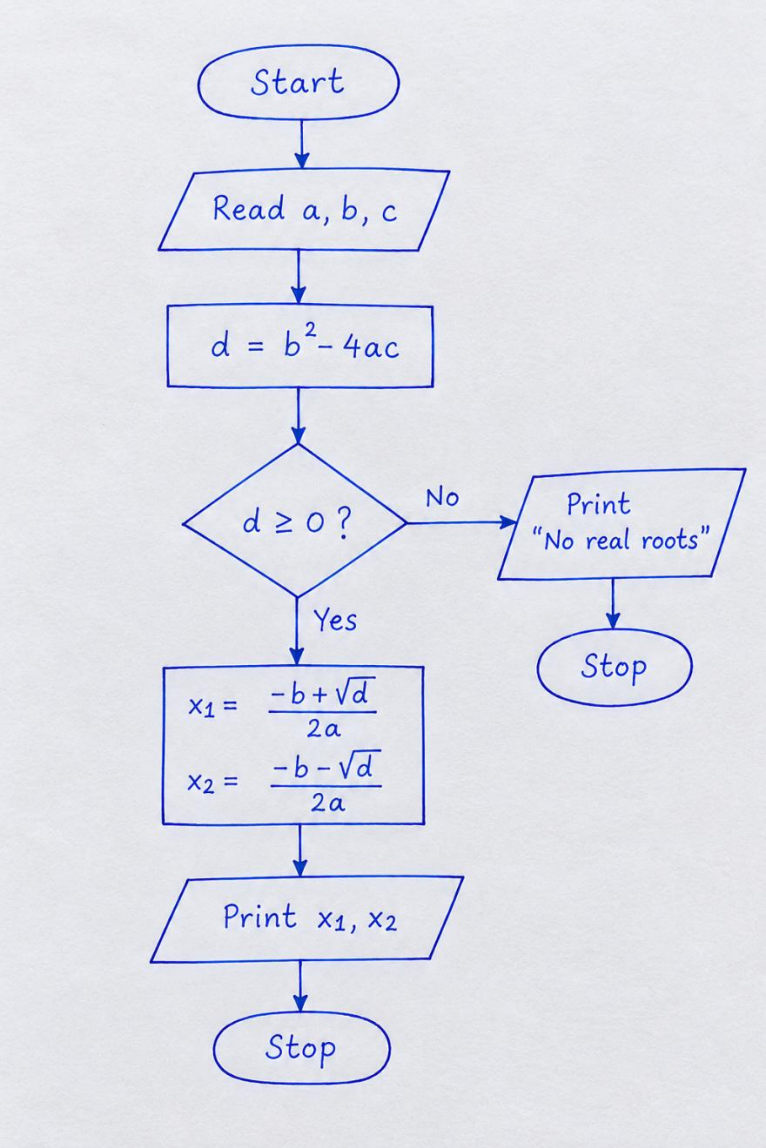
    FOR j = 1 TO i
```

PRINT j

PRINT new line

STOP

6.Flowchart



7.Python Program

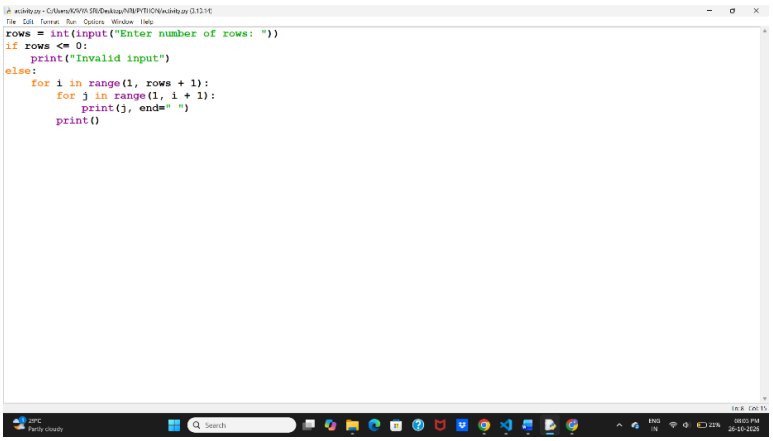
```
rows = int(input("Enter number of rows: "))

if rows <= 0:
   print("Invalid input")
else:
   for i in range(1, rows + 1):
```

```
    for j in range(1, i + 1):
        print(j, end=" ")
    print()
```

8.Test cases

Test Case	Input	Expected Output
1	3	Number pattern with 3 rows
2	5	Number pattern with 5 rows
3	0	Invalid input
4	-2	Invalid input





9.Sample Output:

Enter number of rows: 3

1

1 2

1 2 3

10. Conclusion

The program generates numeric patterns using nested loops and validates the number of rows entered by the user.

CASE STUDY 27: Repeated Transaction Detection

1. Problem Statement

To develop a Python program that identifies repeated transactions in a list of transactions.

2. Objectives

- To detect duplicate transactions.
- To use lists and loops in Python.
- To identify repeated transaction amounts.

3. Assumptions

- The user enters the number of transactions.
- Transaction amounts are entered as numbers.
- If an amount appears more than once, it is considered repeated for this simple example.

4. Algorithm

1. Start.
2. Input the number of transactions.
3. Read each transaction amount into a list.
4. Check whether any amount is repeated.

5. Display the repeated amounts or a message if there are none.
6. stop

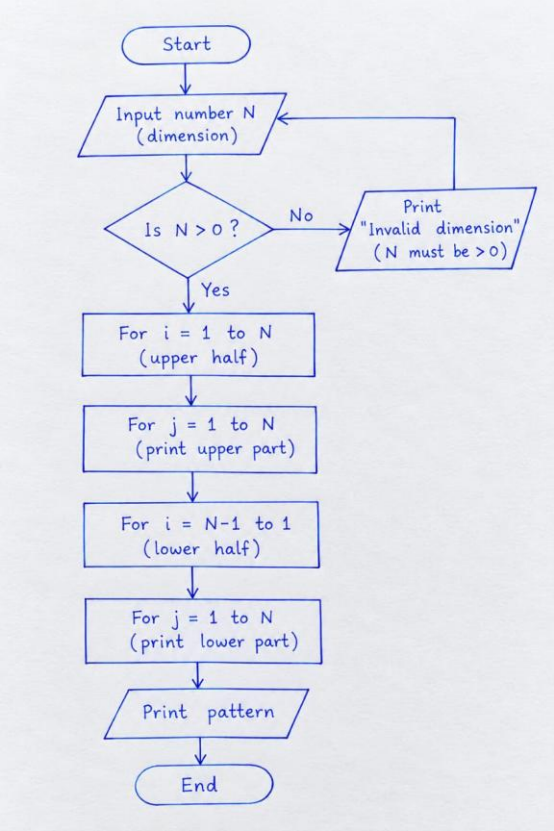
5.pseudo code

```
START
INPUT rows

IF rows <= 0
  PRINT "Invalid input"
ELSE
  FOR i = 1 TO rows
    FOR j = 1 TO i
      PRINT j
    PRINT new line

STOP
```

6. Flowchart



7. Python Program

```
transactions = [
   ("T101", 500),
   ("T102", 2000),
   ("T101", 500),
   ("T103", 7000)
]

suspicious = []
ids = []
limit = 5000

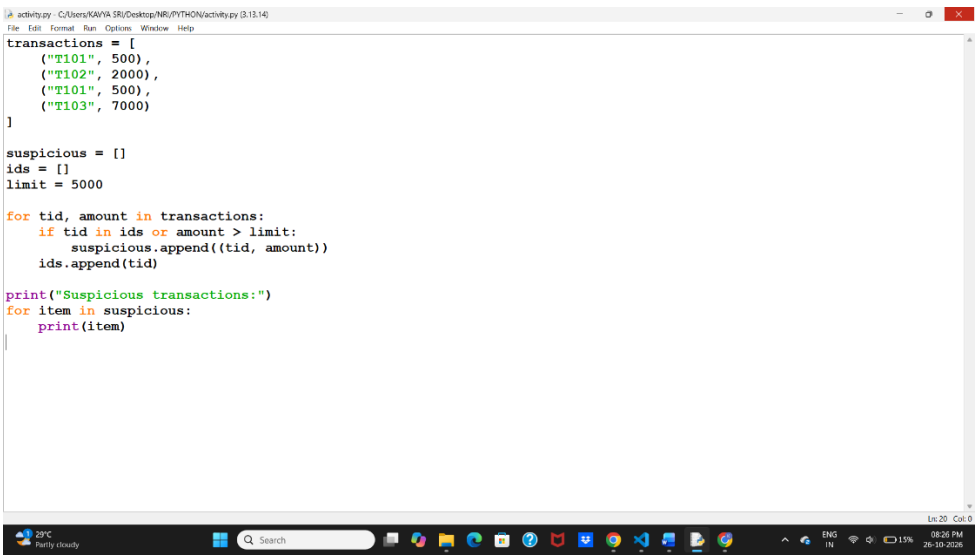
for tid, amount in transactions:
   if tid in ids or amount > limit:
```

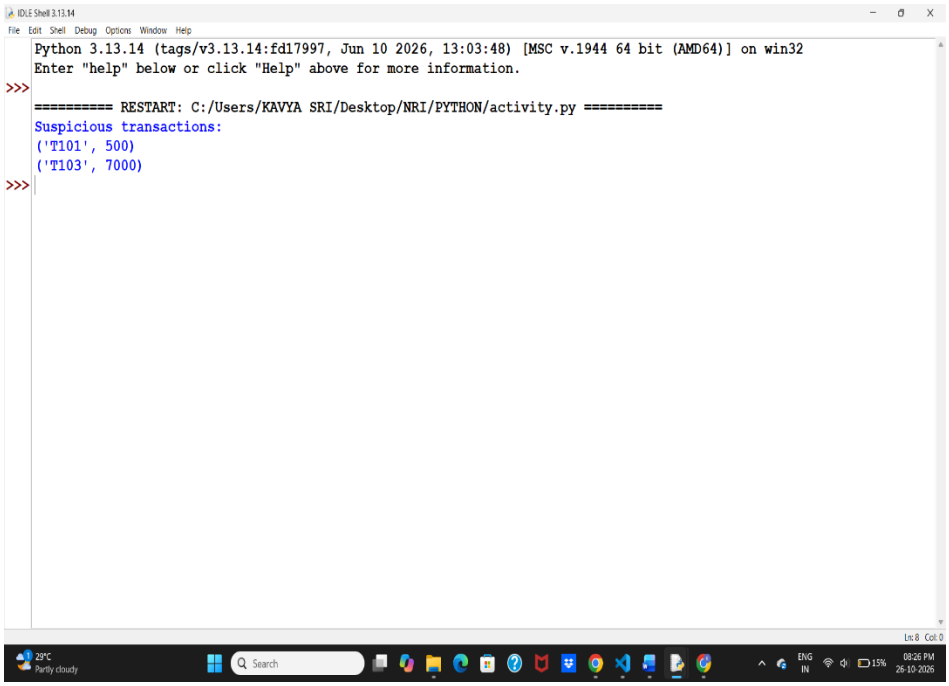
```
        suspicious.append((tid, amount))
    ids.append(tid)

print("Suspicious transactions:")
for item in suspicious:
    print(item)
```

8. Test Cases and Output

Test Case	Transaction	Expected Result
1	T101, 500 (first entry)	Normal transaction
2	T101, 500 (repeated ID)	Suspicious
3	T103, 7000	Suspicious: amount exceeds limit
4	T104, 300	Normal transaction





9.sample Output:

Suspicious transactions:

('T101', 500)

('T103', 7000)

10. Conclusion

The program identifies duplicate transaction IDs and transactions exceeding the specified amount limit using lists and loops.

Think

To pick up a draggable item, press the space bar. While dragging, use the arrow keys to move the item. Press space again to drop the item in its new position, or press escape to cancel.

CASE STUDY 28: Repeated Transaction Detection

1. Problem Statement

To develop a Python program that identifies repeated transactions in a list of transactions.

2. Objectives

- To detect duplicate transactions.
- To use lists and loops in Python.
- To identify repeated transaction amounts.

3. Assumptions

- The user enters the number of transactions.
- Transaction amounts are entered as numbers.
- If an amount appears more than once, it is considered repeated for this simple example.

4. Algorithm

1. Start.
2. Input the number of transactions.
3. Read each transaction amount into a list.
4. Check whether any amount is repeated.
5. Display the repeated amounts or a message if there are none.
6. Stop.

5.pseudo code

START

SET transactions list

SET suspicious list

SET ids list

SET limit = 5000

FOR each transaction

READ transaction ID and amount

IF ID already exists OR amount > limit

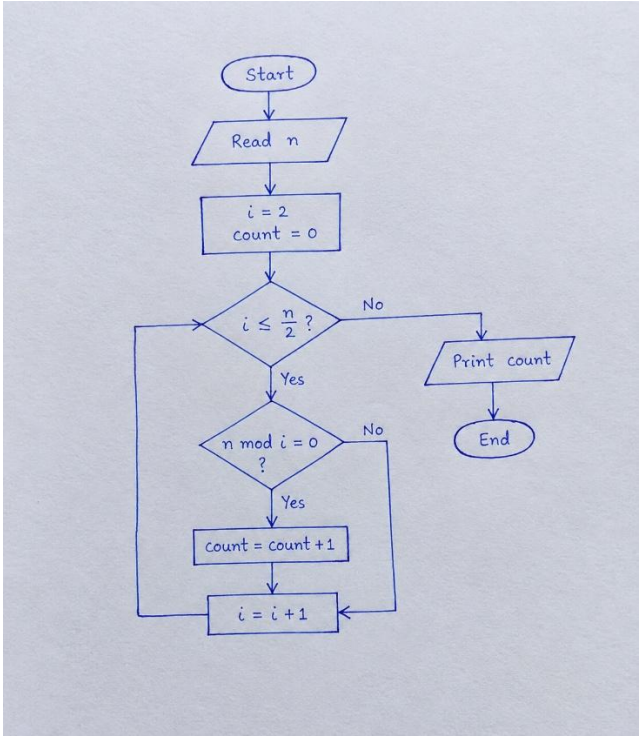
ADD transaction to suspicious list

ADD ID to ids list

PRINT suspicious transactions

STOP

6. Flowchart



7.Python Program

transactions = [

("T101", 500),

("T102", 2000),

```
    ("T101", 500),
    ("T103", 7000)
]

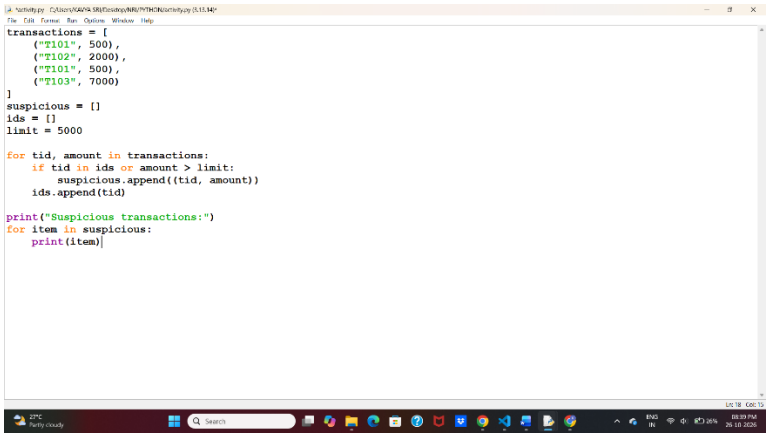
suspicious = []
ids = []
limit = 5000

for tid, amount in transactions:
    if tid in ids or amount > limit:
        suspicious.append((tid, amount))
    ids.append(tid)

print("Suspicious transactions:")
for item in suspicious:
    print(item)
```

8. Test Cases and Output

Test Case	Transaction	Expected Result
1	T101, 500 (first entry)	Normal transaction
2	T101, 500 (repeated ID)	Suspicious
3	T103, 7000	Suspicious: amount exceeds limit
4	T104, 300	Normal transaction



```
transactions = [
    ("T101", 500),
    ("T102", 2000),
    ("T101", 500),
    ("T103", 7000)
]
suspicious = []
ids = []
limit = 5000

for tid, amount in transactions:
    if tid in ids or amount > limit:
        suspicious.append((tid, amount))
    ids.append(tid)

print("Suspicious transactions:")
for item in suspicious:
    print(item)
```



```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/KAVYA SRI/Desktop/NRI/PYTHON/activity.py ==========
Suspicious transactions:
('T101', 500)
('T103', 7000)
>>>
```

9.Sample Output

Suspicious transactions:

('T101', 500)

('T103', 7000)

10. Conclusion

The program identifies duplicate transaction IDs and transactions exceeding the specified amount limit using lists and loops.

CASE STUDY 29: Password Policy Validation Engine

1. Problem Statement

To develop a Python program that checks whether a password satisfies specified rules, such as minimum length, character types, repeated characters, and prohibited words.

2. Objectives

- To validate password rules.
- To use strings and conditional statements.
- To identify and display password errors.

3. Assumptions

- Minimum password length is 8 characters.
- Password must contain uppercase, lowercase, and numeric characters.
- Password must not contain the word password.
- Three or more consecutive identical characters are not allowed.

4. Algorithm

1. Start.
2. Input a password.
3. Check its length.
4. Check uppercase, lowercase, and numeric characters.
5. Check repeated characters and prohibited words.
6. Display whether the password is valid or invalid.
7. Stop.

5. Pseudocode

```
START

INPUT password

SET errors = 0


IF length < 8

   PRINT "Too short"
```

IF uppercase is missing

PRINT "Uppercase missing"

IF lowercase is missing

PRINT "Lowercase missing"

IF digit is missing

PRINT "Digit missing"

IF password contains "password"

PRINT "Prohibited word found"

IF three consecutive characters are same

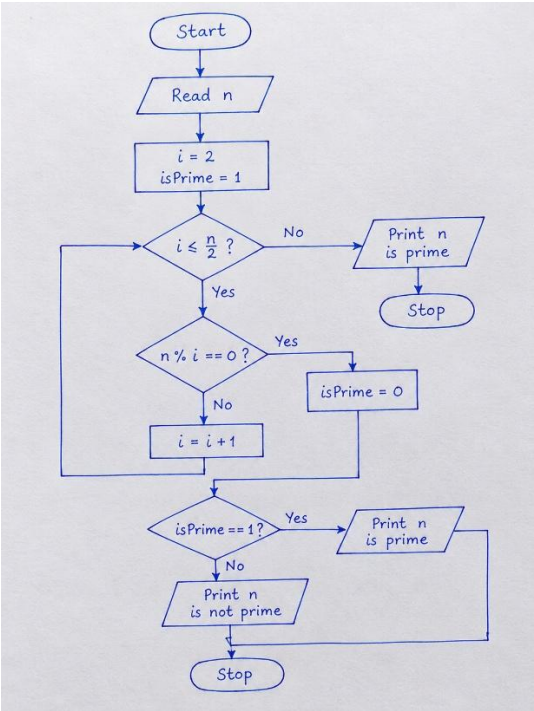
PRINT "Repeated characters found"

IF no errors

PRINT "Valid password"

STOP

6. Flowchart



7.Python Program

```
p = input("Enter password: ")
errors = []

if len(p) < 8:
  errors.append("Minimum 8 characters required")

if not any(c.isupper() for c in p):
  errors.append("Uppercase letter missing")

if not any(c.islower() for c in p):
  errors.append("Lowercase letter missing")

if not any(c.isdigit() for c in p):
  errors.append("Digit missing")

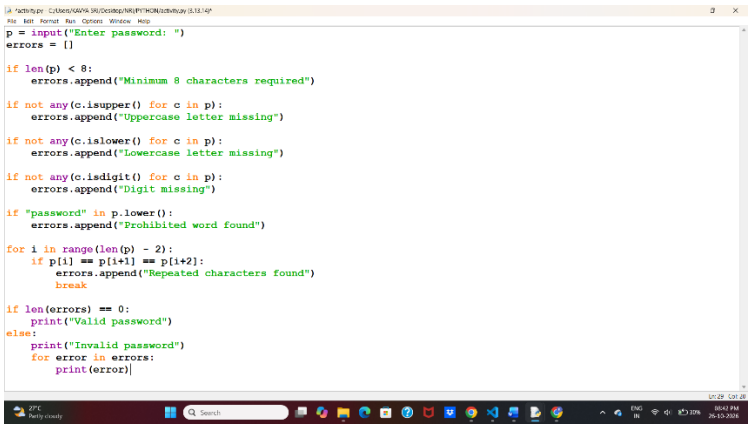
if "password" in p.lower():
  errors.append("Prohibited word found")

for i in range(len(p) - 2):
  if p[i] == p[i+1] == p[i+2]:
    errors.append("Repeated characters found")
    break

if len(errors) == 0:
  print("Valid password")
else:
  print("Invalid password")
  for error in errors:
    print(error)
```

8. Test Cases and Output

Test Case	Input	Expected Output
1	Hello123	Valid password
2	abc12	Invalid — too short and missing uppercase
3	PASSWORD123	Invalid — lowercase missing and prohibited word found
4	Abcdddd123	Invalid — repeated characters found



```
p = input("Enter password: ")
errors = []

if len(p) < 8:
    errors.append("Minimum 8 characters required")

if not any(c.isupper() for c in p):
    errors.append("Uppercase letter missing")

if not any(c.islower() for c in p):
    errors.append("Lowercase letter missing")

if not any(c.isdigit() for c in p):
    errors.append("Digit missing")

if "password" in p.lower():
    errors.append("Prohibited word found")

for i in range(len(p) - 2):
    if p[i] == p[i+1] == p[i+2]:
        errors.append("Repeated characters found")
        break

if len(errors) == 0:
    print("Valid password")
else:
    print("Invalid password")
    for error in errors:
        print(error)
```



```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/KAVYA SRI/Desktop/NRI/PYTHON/activity.py ==========
Enter password: abc 123
Invalid password
Minimum 8 characters required
Uppercase letter missing
>>>
```

9.Sample Output:

Enter password: abc12

Invalid password

Minimum 8 characters required

Uppercase letter missing

10. Conclusion

The program validates passwords using string operations, loops, and conditional statements and displays the rules that are not satisfied.

CASE STUDY 30: Student Email and Identifier Validation

1. Problem Statement

To develop a Python program that validates student email addresses and registration IDs based on specified formatting rules and identifies invalid entries.

2. Objectives

- To validate student email addresses.
- To check registration ID formats.
- To identify missing or incorrect components.

3. Assumptions

- Email must contain @ and end with .com.
- Registration ID must start with NRI followed by digits.
- Invalid entries should display an error message.

4. Algorithm

1. Start.
2. Input student email and registration ID.
3. Check the email format.
4. Check the registration ID format.
5. Display valid or invalid messages.
6. Stop.

5. Pseudocode

```
START

INPUT email, reg_id


IF email contains "@" and ends with ".com"
   PRINT "Valid email"
ELSE
   PRINT "Invalid email"
```

IF reg_id starts with "NRI" and remaining characters are digits

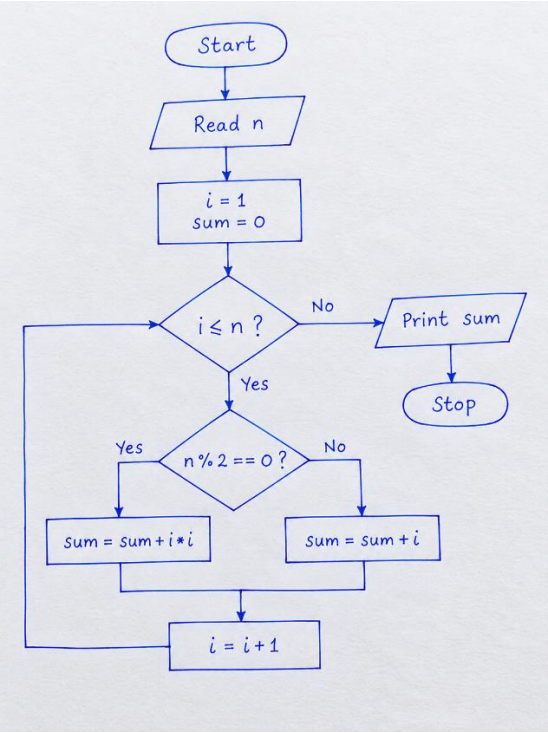
PRINT "Valid registration ID"

ELSE

PRINT "Invalid registration ID"

STOP

6. Flowchart



7.Python Program

email = input("Enter student email: ")

reg_id = input("Enter registration ID: ")

if "@" in email and email.endswith(".com"):

print("Valid email")

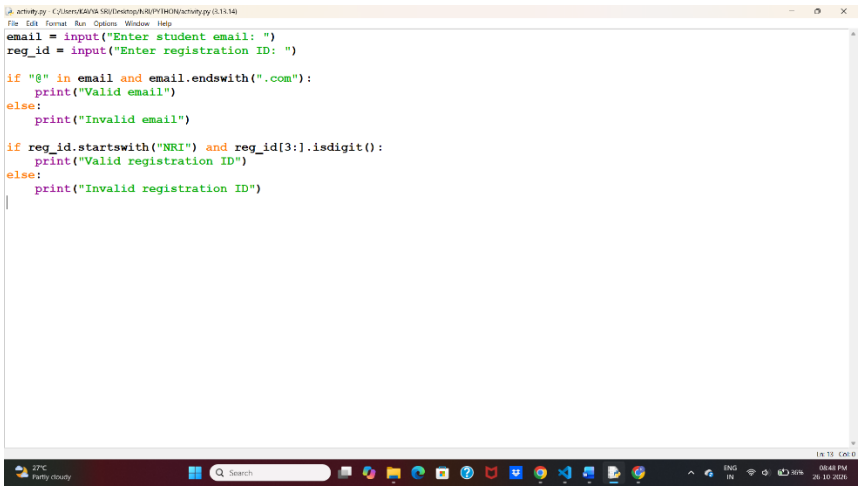
else:

print("Invalid email")

```
if reg_id.startswith("NRI") and reg_id[3:].isdigit():
    print("Valid registration ID")
else:
    print("Invalid registration ID")
```

8. Test Cases and Output

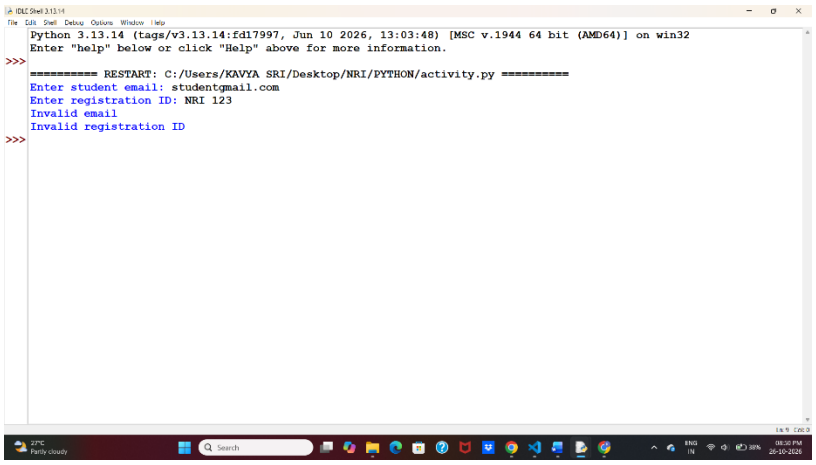
Test Case	Email	Registration ID	Expected Result
1	student@gmail.com	NRI123	Both valid
2	studentgmail.com	NRI123	Invalid email
3	student@gmail.com	ABC123	Invalid registration ID
4	student@gmail.com	NRIABC	Invalid registration ID



```
email = input("Enter student email: ")
reg_id = input("Enter registration ID: ")

if "@" in email and email.endswith(".com"):
    print("Valid email")
else:
    print("Invalid email")

if reg_id.startswith("NRI") and reg_id[3:].isdigit():
    print("Valid registration ID")
else:
    print("Invalid registration ID")
```



```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/KAVYA SRI/Desktop/NRI/PYTHON/activity.py ==========
Enter student email: studentgmail.com
Enter registration ID: NRI 123
Invalid email
Invalid registration ID
>>>
```

9.Sample Output

Enter student email: student@gmail.com

Enter registration ID: NRI123

Valid email

Valid registration ID

10. Conclusion

The program validates student email addresses and registration IDs using string methods and conditional statements. It displays whether each entry is valid or invalid.